HOW TO READ YOUR PROJECT OUTPUT USING OSI THEORY

(Textbook-level explanation mapped to your emulator)

IMPORTANT FIRST STATEMENT (say this in class)

> "This project is an educational emulator of the OSI model. It does not transmit real packets, but it accurately demonstrates **encapsulation, segmentation, framing, line encoding, and modulation**, which are the exact concepts described in networking textbooks."

This sentence alone sets the frame correctly.

LAYER 7 — APPLICATION LAYER

Output shown:

```
Layer 7 — Application
HEllo
```

Textbook definition

> The Application layer provides network services directly to user applications.

What this means **theoretically**

- No networking happens here
- No headers
- No addressing
- No reliability

The data here is called:

> **Application Data / Application Payload**

In your project

- `HEllo` is **pure payload**

- This corresponds to:
    - HTTP request body
    - FTP command
    - SMTP message

Important exam point

❗ Application layer **does not know**:

- IP address
- MAC address
- TCP/UDP
- Bits

That is why **no headers appear here**.

LAYER 6 — PRESENTATION LAYER

Output shown:

```
Plain Encoded Data:
HEllo
```

(or encrypted version if enabled)

WHAT THIS LAYER DOES IN THEORY

Textbook:

> The Presentation layer is responsible for translation, encryption, and compression.

Translation (Encoding)

- Converts data into a **common format**
- Example:
    - ASCII
    - UTF-8
    - Base64

In your project:

- ASCII → Base64 (when encryption is enabled)

Encryption

- Protects confidentiality
- Happens **before** transport
- Does NOT affect routing

Important clarification for teachers

❗ Encryption **does not change the meaning**, only representation.

So:

```
HEllo  →  T0Jra2g=
```

Payload meaning is same, representation is different.

WHAT THIS LAYER DOES NOT DO

- Does NOT add ports
- Does NOT split data
- Does NOT add addresses

This is why the output is still **one continuous block**.

LAYER 5 — SESSION LAYER

Output shown:

```
Session ID: 579B76
Status: active
```

THEORY (what books actually mean)

Textbook:

> The Session layer establishes, manages, and terminates sessions between applications.

A **session** is:

- A logical communication context
- Not physical
- Not packet-based

Real-world examples

- Login session
- Video call session
- Database connection session

IN YOUR PROJECT

The Session ID:

```
579B76
```

Represents:

- A **logical identifier**
- Used to group all communication
- Helps with:
  - Synchronization
  - Recovery
  - Dialog control

❗ Session layer **does not add headers to packets** That's why you see **no packet header here** — only metadata.

This is **correct OSI behavior**.

LAYER 4 — TRANSPORT LAYER (VERY IMPORTANT)

Output:

```
Protocol: TCP
Header:
```

```
{
  "srcPort": 7097,
  "dstPort": 80,
  "seq": 1000,
  "ack": 0,
  "flags": ["SYN"]
}
Segments:
#0: HEll
#1: o
```

CORE THEORY

Textbook:

> The Transport layer provides end-to-end communication, segmentation, flow control, and reliability.

1️⃣ PORT NUMBERS (VERY IMPORTANT FOR EXAMS)

```
srcPort: 7097
dstPort: 80
```

Meaning

- IP identifies **machines**
- Port identifies **processes/applications**

Port 80 = HTTP server Random source port = client process

❗ Without ports, a computer cannot know **which application** should receive data.

2️⃣ TCP HEADER (EXACTLY WHAT YOU STUDIED)

```
seq: 1000
ack: 0
flags: SYN
```

What this means theoretically

- TCP is **connection-oriented**
- Communication starts with **3-way handshake**
- `SYN` flag = request to establish connection

This output literally shows:

> **Step 1 of TCP handshake**

This is **not cosmetic**, this is textbook TCP.

3️⃣ SEGMENTATION (THIS IS CRITICAL)

```
Segments:
#0: HEll
#1: o
```

Definition

> Segmentation is the process of dividing application data into smaller units at the transport layer.

Why segmentation exists

- Networks have size limits (MTU)
- Smaller pieces are easier to retransmit
- TCP can resend **only lost segments**

❗ This is **not framing** ❗ This is **not packetization** This is **segmentation**

LAYER 3 — NETWORK LAYER

Output:

```
IP Header:
{
  "version": 4,
  "ihl": 5,
  "ttl": 64,
  "protocol": "TCP",
  "srcIP": "192.168.1.207",
  "dstIP": "8.8.8.168"
}
```

CORE THEORY

Textbook:

> The Network layer provides logical addressing and routing.

WHAT THIS HEADER MEANS (FIELD BY FIELD)

`version: 4`

- IPv4 protocol

`ihl: 5`

- Internet Header Length
- Size of IP header

`ttl: 64`

- Time To Live
- Prevents infinite routing loops

`protocol: TCP`

- Tells receiver which transport protocol to hand data to

❗ Routers read **ONLY IP header**, nothing above it.

PACKETIZATION

```
Packets:
#0 DATA=HEll
#1 DATA=o
```

Definition

> Packetization is the process of encapsulating transport segments inside IP packets.

Each segment → one packet

This is **exactly correct networking behavior**.

LAYER 2 — DATA LINK LAYER (VERY EXAM-IMPORTANT)

Output:

```
Frame Header:
SRC MAC: 26:51:c5:2f:53:1c
DST MAC: 8b:1b:88:29:3e:b4
```

CORE THEORY

Textbook:

> The Data Link layer provides framing, physical addressing, and error detection.

MAC ADDRESSES (DO NOT CONFUSE WITH IP)

- MAC = physical address
- Used only in **local network**
- Switches use MAC, not IP

So before data goes on wire:

- IP packet is wrapped into a **frame**
- MAC addresses are added

ERROR DETECTION (THIS IS WHERE PARITY / CRC LIVES)

```
Frames:
#0 DATA=HEll CHECKSUM=1100110000110100
#1 DATA=o CHECKSUM=0000000101100010
```

THIS ANSWERS YOUR QUESTION ABOUT PARITY

Parity / CRC:

- Belong to **Data Link layer**
- Used for **error detection**
- Implemented as **frame trailer**

❗ The parity bits are NOT inside data ❗ They are appended as **trailer**

In real Ethernet:

```
[HEADER][DATA][FCS]
```

Your emulator shows:

```
DATA=... CHECKSUM=...
```

That is **correct abstraction**.

LAYER 1 — PHYSICAL LAYER (NRZ, MANCHESTER, ASK, FSK)

This is where your **NRZ confusion ends**.

VERY IMPORTANT STATEMENT (MEMORIZE THIS)

> **NRZ, Manchester, ASK, FSK, and PSK belong exclusively to the Physical Layer.**

They:

- Do NOT appear in headers
- Do NOT appear in frames
- Do NOT appear as text

WHAT NRZ ACTUALLY IS

NRZ = **Line Encoding**

- Converts bits → voltage levels
- Example:
  - 1 → high voltage
  - 0 → low voltage

This happens **after framing**, **before modulation**.

WHAT MODULATION IS (ASK, FSK, PSK)

Modulation:

- Converts encoded bits → waveforms

- Used for:
  - Wireless
  - Copper
  - Optical transmission

Your canvas graph is showing:

> **The final physical signal**, not data.

FINAL BOOK-STYLE SUMMARY (READ THIS)

> In this project, application data is first generated at the Application layer. The Presentation layer optionally encrypts and encodes the data. The Session layer maintains a logical communication context. The Transport layer segments the data and adds reliability using TCP. The Network layer packetizes the segments and adds IP addresses for routing. The Data Link layer frames the packets, adds MAC addresses, and performs error detection using parity or CRC. Finally, the Physical layer converts the bits into electrical or electromagnetic signals using line encoding and modulation techniques.